

N-ary Trees



WATERLOO
ENGINEERING

Douglas Wilhelm Harder, M.Math. LEL

Department of Electrical and Computer Engineering
University of Waterloo
Waterloo, Ontario, Canada

ece.uwaterloo.ca

dwharder@alumni.uwaterloo.ca

© 2006-2013 by Douglas Wilhelm Harder. Some rights reserved.



Outline

This topic quickly looks at a generalization of a binary tree, where each node has up to N children

- Definition
- Perfect N -ary trees
- Complete N -ary trees
- Implementation using templates



5.4.1

N-ary Trees

One generalization of binary trees are a class of trees termed *N*-ary trees:

- A tree where each node had *N* sub-trees, any of which may be empty trees

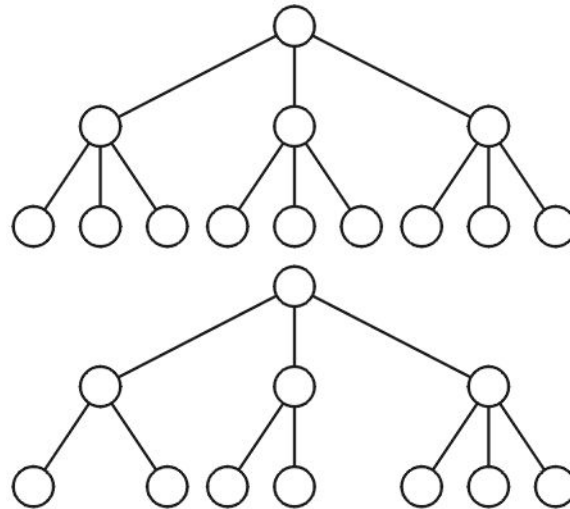


5.4.1

Ternary Trees

Examples of a ternary (3-ary) trees

- We don't usually indicate empty sub-trees

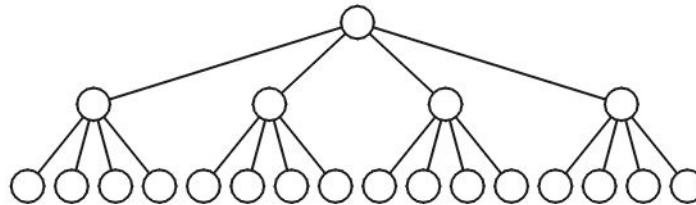




5.4.1

Quaternary Trees

Example of a perfect quaternary (4-ary) tree



Under certain conditions, the number of operations required may be slightly fewer with quaternary trees (e.g., with heaps)

- The asymptotic behaviour is similar



5.4.2.1

Perfect *N*-ary Trees

Each node can have *N* children

The number of nodes in a perfect *N*-ary tree of height *h* is

$$1 + N + N^2 + N^3 \cdots + N^h$$

This is a geometric sum, and therefore, the number of

nodes is $n = \sum_{k=0}^h N^k = \frac{N^{h+1} - 1}{N - 1}$



5.4.2.2

Perfect *N*-ary Trees

Solving this equation for h , a perfect N -ary tree with n nodes has a height given by

$$h = \log_N (n(N - 1) + 1) - 1$$



5.4.2.2

Perfect *N*-ary Trees

We note, however, that $\log_N(nN) - 1 = \log_N(n)$, hence, we may postulate that $\log_N(n)$ should be an equally good approximation

$$\begin{aligned} \lim_{n \rightarrow \infty} \frac{\log_N(n(N-1) + 1) - 1}{\log_N(n)} &= \lim_{n \rightarrow \infty} \frac{\frac{N-1}{(n(N-1) + 1) \ln(N)}}{\frac{1}{n \ln(N)}} \\ &= \lim_{n \rightarrow \infty} \frac{n(N-1)}{nN - n + 1} = \lim_{n \rightarrow \infty} \frac{N-1}{N-1} = 1 \end{aligned}$$



5.4.2.3

N-ary Trees

Recall that the height for perfect binary tree is approximately $\lg(n)$

Therefore, if the height of an *N*-ary tree is $\log_N(n)$, the ratio of the heights is:

$$\frac{\log_2(n)}{\log_N(n)} = \frac{\log_2(n)}{\log_2(n)/\log_2(N)} \\ = \log_2(N)$$



5.4.2.3

N-ary Trees

Therefore:

- The height of a perfect quaternary tree is approximately $\frac{1}{2}$ the height of a perfect binary tree with approximately the same number of nodes
- The height of a perfect 16-ary tree is approximately $\frac{1}{4}$ the height of a perfect binary tree with the same number of nodes



5.4.2.4

Complete *N*-ary Trees

A complete *N*-ary tree has height

$$h = \left\lfloor \log_N \left((N-1)n \right) \right\rfloor$$

Like complete binary trees, complete *N*-ary trees can also be stored efficiently using an array:

- Assume the root is at index 0
- The parent of a node with index *k* is at location $\left\lfloor \frac{k-1}{N} \right\rfloor$
- The children of a node with index *k* are at locations

$$kN + j$$

for $j = 1, \dots, N$



5.4.3

Implementation of *N*-ary Trees

The “obvious” implementation of *N*-ary trees may be something like:

```
# include <algorithm>

template <typename Type>
class Nary_tree {
    private:
        Type element;
        int N;
        Nary_tree **children;

    public:
        Nary_tree( Type const &, int = 2 );
        // ...
};

template <typename Type>
Nary_tree<Type>::Nary_tree( Type const &e, int n ):
    element( e ),
    N( std::max( 2, n ) ),
    children( new *Nary_tree[N] ) {
    for ( int i = 0; i < N; ++i ) {
        children[i] = nullptr;
    }
}
```




Implementation of *N*-ary Trees

Problems with this implementation:

- Requires dynamic memory allocation
- A destructor is required to delete the memory
- No optimizations possible
- Dynamic memory allocation may not always be available (embedded systems)

Solution?

- Specify *N* at compile time...



5.4.3 *N*-ary Trees with Template Parameters

```
#include <algorithm>

template <typename Type, int N>
class Nary_tree {
    private:
        Type element;
        Nary_tree *children[std::max(N, 2)];    // an array of N children
    public:
        Nary_tree( Type const & = Type() )
        // ...
};

template <typename Type, int N>
Nary_tree<Type, N>::Nary_tree( Type const &e ):element( e ) {
    for ( int i = 0; i < N; ++i ) {
        children[i] = nullptr;
    }
}
```



5.4.3 *N*-ary Trees with Template Parameters

Sample code using this class:

```
Nary_tree<int, 4> i4tree( 1975 ); // create a 4-way tree  
std::cout << i4tree.retrieve() << std::endl;
```



5.4.3

N-ary Trees

Because the size of the array (the 2nd template parameter) is specified at compile time:

- The compiler can make certain optimizations
- All memory is allocated at once
 - Possibly even on the stack at compile time
- No destructor required

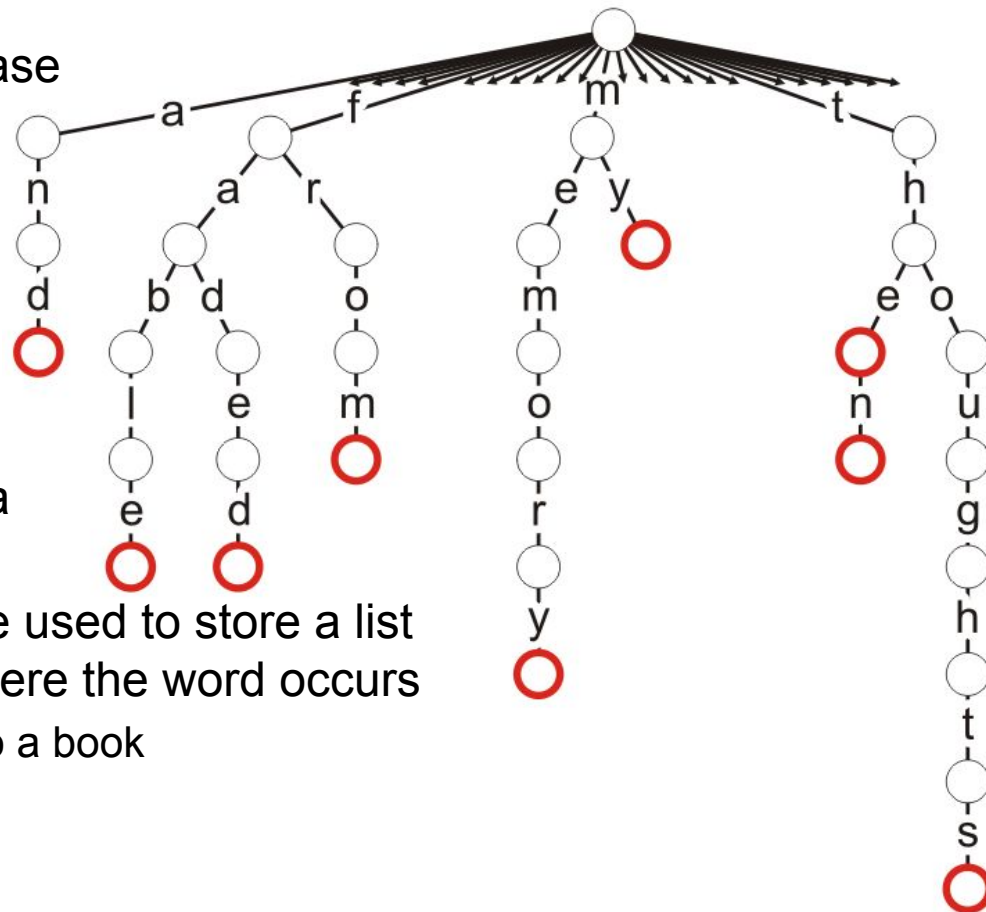


5.4.4

Applications

One application of an 26-ary trees is a *trie* where the root represents the *start* of each valid word, and the different sub-trees represent next letters in valid words

- Consider the words in the phrase
“The fable then faded from my thoughts and memory.”
- All 26 sub-trees are only shown for the root node, but all nodes have 26 sub-trees
- Some nodes are marked as *terminal* indicating the end of a valid word
- These *terminal* points could be used to store a list of all places in a document where the word occurs
 - Consider the ultimate index to a book





Summary

An *N*-ary tree is similar to a binary tree, however, each node can have up to *N* children

A binary tree has a height approximately $\log_2(N)$ times the height of an *N*-ary tree containing the same number of nodes



Reference

- [1] Cormen, Leiserson, and Rivest, *Introduction to Algorithms*, McGraw Hill, 1990, §5.5.3, p.94-96.



Usage Notes

- These slides are made publicly available on the web for anyone to use
- If you choose to use them, or a part thereof, for a course at another institution, I ask only three things:
 - that you inform me that you are using the slides,
 - that you acknowledge my work, and
 - that you alert me of any mistakes which I made or changes which you make, and allow me the option of incorporating such changes (with an acknowledgment) in my set of slides

Sincerely,

Douglas Wilhelm Harder, MMath

`dwharder@alumni.uwaterloo.ca`